

Micro-Trade OS

A Deterministic, Low-Latency RTOS Kernel for the ARM Cortex-M4

Statement of Objectives, Method and Contribution

Target platform: STM32F407 (Cortex-M4) | Baseline for comparison: FreeRTOS-class RTOS

1. The problem

A conventional real-time operating system is built to be *fair* and *responsive on average*. It keeps a periodic timer interrupt running at all times, typically 1 kHz, because that interrupt is what makes preemption possible. The same interrupt is also what makes latency unpredictable: it can land on any instruction of a time-critical task, and the victim pays for the vector fetch, the stack frame, the handler body, a scheduler pass and the return. Averaged over a long run that cost is negligible. As a worst case on a single iteration it can consume the entire timing budget.

For a whole class of systems the average is the wrong thing to optimise. What matters is the promise that *no* iteration will ever exceed a stated bound. This project builds an RTOS kernel in which that bound is a designed, measured and defended property rather than an emergent one.

The thesis in one sentence

Applying three architectural techniques from electronic trading infrastructure, poll-mode scheduling, lock-free cache-partitioned message passing, and tail-latency-first measurement, to a \$10 ARM Cortex-M microcontroller yields determinism that a conventional RTOS configuration on the same silicon does not provide.

2. Objectives

The project has one primary objective and four supporting ones.

Primary

Design, implement and **quantitatively validate** a preemptive fixed-priority RTOS kernel for the Cortex-M4 whose worst-case scheduling jitter and inter-task message latency are measurably lower, and measurably more predictable, than those of a conventional tick-driven, lock-based RTOS running the same workload on the same hardware.

Supporting

- **O1, Poll-mode scheduling.** Provide a task class that can suspend the scheduler tick for a bounded region, so a critical task runs to completion with zero scheduling jitter, and do so *without losing real time*: the suppressed ticks must be reconstructed and replayed when the region closes.
- **O2, Cache-aligned lock-free IPC.** Replace mutex-protected message queues on the hot path with a single-producer/single-consumer ring buffer that is partitioned across cache lines and uses shadow indices, eliminating both lock overhead and false sharing.
- **O3, A benchmarking testbed.** Build the conventional design as a like-for-like control on the same kernel, and measure both under one harness. A claim of improvement is only as good as the baseline it is measured against.
- **O4, Bounded, provable timing.** Retain and verify priority inheritance, response-time schedulability analysis, and an empirical worst-case interrupts-disabled bound, so the measured numbers can be read against a bound claimed in advance.

3. The two new mechanisms

3.1 Poll-mode scheduling

Exchange-located trading systems avoid interruption altogether: the hot thread is pinned, the kernel is kept out of its way, and it polls. Poll-mode scheduling reduces that idea to what a Cortex-M4 can do. A task declared `TASK_CLASS_POLL` may open a *poll section*, and for its duration the kernel guarantees the task is not descheduled.

Three things happen on entry:

- SysTick's `TICKINT` bit is cleared, so the tick cannot fire. `ENABLE` is deliberately left set, so the counter keeps running and no real time is lost. This is what makes the section reversible.
- `rtos_schedule()` stops pending `PendSV` and records a *deferred* switch instead, so nothing can deschedule the task even if an interrupt readies a higher-priority one. This is what makes the guarantee a property of the kernel rather than a side effect of the tick being masked.
- In `RTOS_POLL_EXCLUSIVE` mode, `BASEPRI` is additionally raised so no configurable-priority interrupt runs at all. Faults and NMI stay enabled by design: masking those would trade a jitter problem for a silent-corruption problem.

On exit the kernel replays the suppressed ticks, restores `TICKINT` and `BASEPRI`, and takes the deferred switch. The rest of the task set therefore sees a scheduler that *paused*, not one that lost time. Delays and deadlines remain correct.

Counting what the hardware will not count

SysTick's `COUNTFLAG` saturates, so the hardware cannot report how many reloads occurred while the interrupt was masked. The kernel counts them in software: SysTick counts *down*, so a fresh sample of `VAL` that is *larger* than the previous one means the counter wrapped and one tick was suppressed. The sampling hook is `rtos_poll_expired()`, which the poll loop already calls once per iteration to honour its cycle budget. Where the DWT cycle counter exists (real silicon) the elapsed-cycle count gives an independent second estimate, and the kernel takes the larger of the two: it may run the clock forward, never backward. A late wake-up is a scheduling artefact; a lost tick is a correctness bug.

The honest cost

A poll section is **cooperative**: the kernel cannot end one, only the task can. Determinism inside the section is bought with the programmer's obligation to leave it. The kernel makes that obligation auditable rather than implicit. It records every section's length, every budget overrun and every blocking-call violation, and reports them. A task that blocks inside a section does not hang the system: the kernel force-closes the section, restores the tick and counts the violation.

3.2 Cache-aligned lock-free IPC

The hot-path queue is a wait-free single-producer/single-consumer ring with two structural properties:

- **Cache-line partitioning.** Immutable geometry, producer-owned state (`tail`, `cached_head`) and consumer-owned state (`head`, `cached_tail`) occupy three separate 32-byte regions. Neither side ever dirties a line the other reads on its hot path, so false sharing is eliminated by construction.
- **Shadow indices.** The producer tests for "full" against its own stale copy of the consumer's index and re-reads the real index only when that copy says full. The consumer mirrors this. On a queue that is kept drained, the peer's index leaves the common path entirely.

A claim this project deliberately does not make

The STM32F407 is a Cortex-M4 and **has no data cache**. On this part the padding cannot buy a cache effect, and it is not claimed to. What the alignment buys here is that each index sits alone on a naturally-aligned block, and that the same source stays correct when compiled for a cached Cortex-M7 or any multi-core host, where false sharing is real. The measurable win on the M4 comes from the shadow indices, which remove a load and a dependent compare from the hot path outright. Separating these two effects, rather than attributing both to "cache alignment", is part of the contribution.

4. Method: how the claim is proved

A claim of improvement is only as strong as its control, so the conventional design is **built here**, on the same kernel, rather than quoted from another project. The baseline queue (`queue_baseline.c`) is a mutex-plus-semaphore copying queue of the same shape as FreeRTOS's `xQueueSend/xQueueReceive`, compiled with the same flags at the same clock, and it is given its most favourable honest form: its fast path skips the semaphores entirely. The only variable that differs between subjects is the synchronisation strategy.

Three experiments

#	Experiment	What it isolates	Instrument
1	Context-switch cost, ≥ 10 samples	The kernel's own overhead per switch, as a full distribution	DWT CYCCNT, timed inside PendSV
2	Execution jitter: tick-driven vs poll-mode	Whether suppressing the scheduler tick removes jitter. Identical task set in both phases; the <i>only</i> difference is the poll section	DWT + external logic analyser on PD12/PD13
3	64-byte IPC latency, three queue designs	A vs B isolates the cost of the lock; B vs C isolates the cost of the peer-index load	DWT, batched and interleaved

A 64-byte message is chosen deliberately: it is the size of a market-data tick record and, not coincidentally, exactly two 32-byte cache lines, which makes the alignment question live rather than academic.

Methodological commitments

- **Distributions, not averages.** Determinism is a claim about the *tail*. A mean hides precisely the events the experiments are about, so every result is reported as min / mean / p50 / p99 / p99.9 / max with the histogram that produced it.
- **Two independent views of the jitter claim.** PD13 is held high for the whole measured window, so a logic analyser captures exactly the blocks that produced the printed numbers. The on-chip histogram and the captured waveform must agree.
- **A falsifiable prediction.** Poll mode removes interference; it does not make the loop faster. The quantity comparable across the two phases is the *minimum*, an uninterrupted block, and the two minima must agree up to the cost of the poll machinery itself, which is reported rather than absorbed. What must collapse is the *spread*: a single narrow spike against a multi-modal baseline.
- **Correctness gates the timing.** A latency figure from a queue that drops or reorders messages is worthless, so every queue is verified for round-trip byte integrity, FIFO ordering and full/empty edge behaviour before any timing is reported.
- **Emulator results are labelled as such.** QEMU implements no DWT and is not cycle-accurate; every report states which clock produced it. QEMU verifies function and correctness. The numbers in the report come from the board.

5. Why this matters beyond trading

High-frequency trading motivates the latency targets and the shape of the workload; no claim is made that the platform is a trading system. The underlying capability, *ultra-deterministic OS bypass for resource-constrained embedded systems*. Is what transfers:

Domain	Why jitter is the binding constraint
High-speed robotics and drone swarms	Synchronised flight controllers share telemetry on a fixed cadence. If the RTOS pauses a rotor-synchronisation task to service background work, the airframe drifts within that window. The bound that matters is the worst case, not the average.

Domain	Why jitter is the binding constraint
Automotive drive-by-wire (EV and ADAS)	When a radar return indicates a pedestrian, the braking task must execute within a fixed deadline. Safety cases are written against worst-case response, so a scheduler whose interference is bounded and measured is worth more than one that is merely fast on average.
Industry 5.0 and edge AI	Vibration and acoustic models on-device ingest high-rate SPI/I ² C streams. Interrupt-driven collection spends CPU on per-sample overhead; poll-mode ingest with lock-free hand-off leaves the core free for inference.

6. Academic contribution

Why not an FPGA, or Linux with DPDK, as real trading firms use? Because that is precisely the gap. Enterprise systems reach low latency by bypassing the kernel with frameworks such as DPDK, and those paradigms are rarely applied to deeply embedded, resource-constrained microcontrollers. This project shows that stripping an RTOS down and applying those architectures, poll-mode scheduling, cache-line partitioning, lock-free IPC, tail-latency-first measurement, brings enterprise-grade determinism to a microcontroller costing a few dollars, and it does so with the baseline built alongside so the improvement is measured rather than asserted.

The specific, defensible contributions are:

- A poll-mode scheduling mechanism for Cortex-M that suppresses the scheduler tick *without losing real time*, via software reconstruction of suppressed SysTick reloads cross-checked against the cycle counter. With the cooperative-contract cost made auditable rather than assumed away.
- A kernel-enforced non-preemption interlock, so the zero-jitter guarantee survives an interrupt readying a higher-priority task, instead of depending on the tick merely happening to be masked.
- A like-for-like measurement of lock-based against lock-free IPC on identical hardware and toolchain, separating the cost of the lock from the cost of the memory layout, and reporting the access patterns under which the shadow-index optimisation does *not* pay.
- A statement of the determinism argument as a *structural* property. The lock-free path executes zero interrupt-masking instructions, verifiable directly in the disassembly, rather than as a throughput measurement alone.

7. Results so far (emulation)

The testbed is complete and has been run under emulation. These results establish correctness and the direction and relative size of each effect; they are **not** cycle counts, and the figures for the final report must be regenerated on the board, where the DWT cycle counter makes the clock exact.

Experiment 2, execution jitter

Quantity	Phase A (tick-driven)	Phase B (poll mode)
Peak-to-peak spread	27,806	1
Worst observed block	31,160	3,828
Uninterrupted block (minimum)	3,354	3,827
Distribution shape	multi-modal (4 modes)	single bin, 4000/4000

The histogram is the evidence: Phase A is multi-modal. One mode for blocks that ran uninterrupted, further modes for blocks that absorbed a scheduler slice. While Phase B places all 4,000 samples in a single bin. The worst-case improvement, which is the number a real-time budget is actually written against, is **8.1×**. The gap between the two minima, 473 units per block, *is* the cost of the poll machinery, and is reported next to the benefit rather than absorbed into it.

Experiment 3, 64-byte IPC latency

Correctness gate (round-trip byte integrity, FIFO ordering, full/empty edges): **PASS** for all three queues.

Comparison	Ping-pong (depth 1)	Burst (depth 8)
Lock-free vs mutex queue (A vs B)	1.26×	1.26×
Cache-partitioned vs plain (B vs C)	0.99×	0.99×

Removing the lock is worth about 26% on the hot path, consistently across both access patterns. The cache-line partitioning measures at parity: the Cortex-M4 has no data cache and single-cycle SRAM, so the load the shadow indices remove costs about as much as the branch and the store that replace it. Consistent with Section 3.2, it is a portability and multi-core property and is not claimed as an M4 speed-up.

The determinism result is independent of the timings and is structural: per message the baseline queue enters four critical sections and the lock-free rings enter none, verified directly in the disassembly. Only the baseline can extend another task's worst-case latency.

Head-to-head against FreeRTOS

The objective is stated against FreeRTOS, so FreeRTOS V11.1.0 was built and measured rather than cited. Both images share the boot layer, the measurement code, the cycle clock, the GPIO driver, the compiler and flags, the emulated machine and the workload; FreeRTOS is configured to match ours (8 MHz, 1 kHz tick, preemptive with time-slicing, CLZ task selection, priority-inheritance mutexes) rather than handicapped. The kernel is the only intended variable.

Execution jitter (4000 blocks)	FreeRTOS	Ours, tick-driven	Ours, poll mode
Uninterrupted block (min)	3,161	3,354	3,827
Worst observed block	29,568	31,160	3,828
Peak-to-peak jitter	26,407	27,806	1
Distribution	multi-modal	multi-modal	single bin

Against FreeRTOS, poll mode reduces the worst-case block from 29,568 to 3,828, **7.7×**, and collapses peak-to-peak jitter from 26,407 units to **1**. FreeRTOS provides no mechanism to suppress its scheduler tick for a critical section, so there is no second phase on that side: this is a capability difference, not a tuning difference.

64-byte IPC, per message	Ping-pong	Burst
FreeRTOS xQueueSend / xQueueReceive	9,290	9,445
Our mutex-queue baseline	9,245	9,277
Our lock-free SPSC	7,285	7,317
Our cache-partitioned SPSC	7,340	7,329

The lock-free hot-path queue is **1.28×** faster than FreeRTOS on ping-pong and **1.29×** on burst. Per message, FreeRTOS enters four critical sections and the lock-free ring enters none, so only FreeRTOS can extend the worst-case latency of tasks that never touch the queue. That structural property, verified by disassembly, matters more for a real-time budget than the 1.28×

What the comparison does not support

FreeRTOS is slightly *faster* on the uninterrupted path (3,161 against our 3,354, about 6%). This kernel is not the faster kernel at raw execution and is not claimed to be; the claim is determinism. The context-switch comparison is **inconclusive** under emulation. Both kernels report a minimum of zero, which no real switch costs, and needs the board. Cache-line partitioning measures at parity, as expected on a part with no data cache.

Our own tick-driven mode measures within a few percent of FreeRTOS (27,806 against 26,407 peak-to-peak). That is a validation rather than a disappointment: it establishes that the in-house baseline is an honest stand-in, so the improvement measured against it is real.

8. Deliverables and status

Deliverable	Status
Preemptive fixed-priority kernel, O(1) scheduling, PendSV/SVC switching	Complete, builds warning-free at -O2
Priority-inheritance mutex, verified against a constructed inversion scenario	Complete, blocking time measured and bounded
Poll-mode scheduling (O1), with tick replay and violation accounting	Complete; jitter spread 27,806 → 1 under emulation
Cache-aligned lock-free SPSC ring with shadow indices (O2)	Complete, correctness-verified
Benchmarking testbed: three experiments plus the like-for-like baseline (O3)	Complete; all three experiments run
Schedulability analysis and worst-case interrupts-disabled bound (O4)	Complete
Cycle-accurate measurement on STM32F4 Discovery hardware	Outstanding , requires the board
Logic-analyser capture of the jitter waveform (PD12/PD13)	Outstanding , requires the board

The remaining items, together with work deliberately left out of scope, are listed in docs/OUTSTANDING_WORK.md. Defects found and fixed during development are recorded in docs/DEFECTS_AND_FIXES.md, and the full verification run in docs/TEST_RECORD.md.